Life Cycle Reliability and Safety Engineering
https://doi.org/10.1007/s41872-025-00335-4

ORIGINAL RESEARCH

Enhancing reliability and security in online social networks: intelligent learning models for cybercrime detection

Mohammed Wasid[1] · **Abdullah Yahya Abdullah Amar**[2] · **Shahnawaz Ahmad**[1] · **Jiyaul Mustafa**[3]

Received: 18 February 2025 / Accepted: 2 June 2025
© The Author(s), under exclusive licence to Society for Reliability and Safety (SRESA) 2025

Abstract
The proliferation of online social networks has amplified the risks of cybercrime, including cyberbullying, terrorism-related activities, and online threats. Addressing these challenges requires intelligent systems capable of understanding not only textual content but also the underlying network interactions to ensure platform safety and reliability. This study explores a suite of machine learning (ML) and deep learning (DL) models for cybercrime detection, evaluating their performance using real-world social media data. In this paper, alongside traditional architectures, we propose a new model – Transformer-Augmented Graph Neural Network (T-GNN), which synergistically integrates a transformer's contextual comprehension with a graph neural network's structural understanding. The T-GNN integrates rich semantic feature extraction from text with user interaction modeling through graphs, capturing the essence of social media behavior. Under varying train-test splits and multiple feature extraction strategies, comparative experiments demonstrate that while DL models surpass traditional ML methods, T-GNN continues to achieve the highest accuracy and robustness for all tasks. This research highlights the role of sophisticated models to transform the trustworthiness and safety of digital systems and contribute towards intelligent cybercrime detection systems.

Keywords Cybercrime detection · Transformer · Graph neural network · Deep learning · Machine learning · Natural language processing · Social networks

1 Introduction

The emergence of social networking websites like X (previously Twitter), Instagram, and Facebook has enhanced communication worldwide as people from different geographic regions can now seamlessly communicate and share content with each other. Social networks are utilized by billions of users around the globe, meaning that users create content on a massive scale which provides important information about the prevailing trends, behaviors, and moods of society (Zhang 2025). Social networks have greatly benefitted from the use of artificial intelligence (AI) to offer features like personalized recommendations, ads, automated content moderation, and many more (Teepapal 2025). Specifically, ML and DL techniques have improved the functioning of these platforms by analyzing large amounts of data to find correlations, predict user behavior, and pinpoint outliers. For instance, AI algorithms play an important part in determining what content users see on their feeds, selecting connections for them, and even monitoring user activity for the detection of inappropriate and harmful content.

Regardless, the safety and reliability of online interactions has become a major issue with the emergence of AI tailored social networks. Identity theft, fraud, identity-based cyberbullying, and even the distribution of malware and phishing links are all forms of cybercrime made easier

✉ Shahnawaz Ahmad
shahnawaz98976@gmail.com;
shahnawaz.ahmad@bennett.edu.in

Mohammed Wasid
erwasid@gmail.com

Abdullah Yahya Abdullah Amar
ayaamer@myamu.ac.in

Jiyaul Mustafa
jiyaul.mustafa@bennett.edu.in

[1] School of Computer Science Engineering and Technology, Bennett University, Greater Noida 201310, India

[2] Department of Computer Science, Aligarh Muslim University, Aligarh 202002, India

[3] Department of Mechanical Engineering, Bennett University, Greater Noida 201310, India

Published online: 28 June 2025

by the anonymity and accessibility social media provides (Nabeel and Iqbal 2025). The ease with which individuals can be harassed online without revealing their identity allows for acts of cyberbullying to inflict great psychological damage upon unsuspecting victims. Likewise, social media platforms provide dangerous content with the ability to promote extremist ideals without any check. The scope of these social crimes coupled with their advanced techniques pose a threat to the safety of users on social media and social networks, which illustrates the need for infrastructure aimed at detecting these crimes to protect users and maintain the sanctity of the platform (Aksoy et al. 2025).

As an enabler of cybercrime and as a mitigation tool, AI has performed both functions zealously (Tarade and Das 2025). On the other side of the coin, AI-based detection systems use ML and DL algorithms to detect, exploit, and neutralize cyber threats in real-time. Prior research has utilized Random Forest (RF), Support Vector Machines (SVM) and even Bidirectional Long Short-Term Memory (Bi-LSTM) models to automate the detection and response to cybercrime (Kalpani et al. 2025). While these models have made headway, it is also important to note that their effectiveness towards the changing face of cybercrime on social networks is lacking due to certain unaddressed shortcomings, as listed below:

- Limited Contextual Understanding: Many models, such as traditional ML classifiers (e.g., RF, SVM), rely on handcrafted features like TF-IDF or Bag-of-Words, which fail to capture the nuanced semantics and contextual relationships in social media text.
- High Dimensionality and Informal Language in Social Media Texts: Language and social media data are grammatically rich yet noisy and unstructured. For traditional models, dealing with slang, abbreviation, spelling differences, and semantic variation is a daunting task, resulting in reduced accuracy.
- Inability to Capture Contextual Relationships: Most of the earlier models depend on the bag-of-words or TF-IDF features that do not incorporate contextual understanding. This makes them prone to missing subtler forms of cybercrime, say, indirect threats or sarcastic comments which are deeper.
- Limited Model Comparison and Optimization: Previous work seems to focus on one at a time – either ML or DL – and fail to do a comprehensive comparison. In addition, optimization of hyperparameters and the model architecture are usually omitted too.
- Neglect of Network Interactions: Social networks have a relational nature where user activity, such as replying, retweeting, and mentioning, are interlinked in intricate networks. Most existing models analyze networks with a focus on textual content while neglecting textual patterns that could point to sophisticated malicious actions like propaganda via bots or organized cybercrime operations.

This study approaches the detection gaps by highlighting the creation of sophisticated cybercrime detection systems using various ML and DL models, hereafter called intelligent learning models in this paper. The major aim is to construct intelligent systems able to sift through extensive social media datasets and accurately detect and flag malicious or unauthorized content.

In this regard, we apply an advanced text mining and NLP pipeline to perform data cleansing, transformation, and extraction to obtain important features from raw text data. Also, we analyze several Naïve Bayes, SVM, K-Nearest Neighbors, RF, and Logistic Regression classifiers as a part of feature engineering using TF-IDF, Bag of Words, and TF-IDF with unigrams and bigrams.

Besides these, we introduce different DL models: CNN, LSTM, Bi-LSTM and Gated Recurrent Unit (GRU). Furthermore, we add a new model called Transformer-Augmented Graph Neural Network (T-GNN) which synthesizes the expressive power of transformers with the structural reasoning capability of graph neural networks. T-GNN applies DistilBERT for understanding subtleties of context in the text and uses Graph Attention Networks (GAT) for modeling user-content interactions, which gives a better understanding of cybercriminal activities in networked data. Besides improving understanding of the text, this model also captures relational patterns between users, which greatly enhances classification accuracy.

We can state the contribution of this work as follows:

- We propose a unified hybrid framework for the detection of cybercrime events that integrates traditional ML, new-age DL, and graph-based neural models for improved performance.
- We build a T-GNN model that extracts contextual and structural information from social media data.
- We run comprehensive tests on different subsets of data using multiple embedding methods (Word2Vec, FastText) and assess performance in terms of accuracy, precision, recall, and F1-score.

The paper is arranged as such: Sect. 2 explores existing studies, Sect. 3 lays out the main methodology, Sect. 4 explains the experiment settings, Sect. 5 reports the results and Sect. 6 continues with ideas for future work.

2 Literature survey

The proliferation of online social networks has heightened the need for effective cybercrime detection systems to counter threats such as cyberbullying, terrorism-related activities, fraud, and identity theft. Recent research has leveraged ML and DL techniques, combined with advanced feature extraction and network analysis methods, to classify malicious content on platforms like X (formerly Twitter), Facebook, and Reddit. This section reviews key studies, highlighting their methodologies, datasets, and findings, while identifying limitations that motivate the proposed T-GNN approach.

ML techniques have been widely adopted for cybercrime detection due to their interpretability and efficiency with structured data. Balakrishnan et al. (2020) employed classifiers such as J48, RF, and NB to detect cyberbullying on Twitter, categorizing tweets into routine aggressors, bullying, and spammers. Their study achieved moderate accuracy but was constrained by a small dataset and limited classification labels, reducing generalizability. Nahar et al. (2014) proposed a semi-supervised fuzzy SVM for cyberbullying detection, utilizing an augmented training set from streaming data. While effective for dynamic data, their approach struggled with initial small datasets, impacting early-stage performance. Dadvar et al. (2018) explored hate speech detection across multiple platforms (Wikipedia, Reddit, YouTube, Twitter), analyzing 197,566 comments with ML models like LR and SVM. Their findings emphasized the importance of feature selection but noted challenges in handling cross-platform linguistic variations.

DL models, particularly recurrent and convolutional architectures, have shown superior performance in capturing sequential and contextual patterns in text. Zhao et al. (2017) developed a semantic-enhanced marginalized denoising auto-encoder combined with an SVM classifier for cyberbullying detection, achieving robust results by generating informative content representations. However, their model's reliance on SVM for classification limited its ability to fully exploit DL's end-to-end learning capabilities. Dewani et al. (2021) extended SVM-based cyberbullying detection using Word2Vec embeddings on a Roman Urdu dataset, reporting an F1-score of 0.78. Their approach was hindered by the lack of contextual embeddings, reducing effectiveness for nuanced text. Wendi et al. (2023) compared Bi-LSTM, Recurrent Neural Networks (RNN), and CNN for cyberbullying detection, finding Bi-LSTM to be the most effective due to its bidirectional context modeling. Likewise, Agarwal and others, as in reference (Murshed et al. 2022), proposed a modified RNN with class weighting and under sampling that exceeded the performance of basic LSTM models while requiring severe tuning of hyperparameters.

Transforms, as a recent technology, provide NLP applications with breathtaking contextual capabilities. As stated in Dadvar and Eckert's work in reference (Dadvar and Eckert 2020), the BERT-based model achieved great results in detecting cyberbullying on social media by making use of the bidirectional embeddings it offers. However, BERT's computational complexity posed challenges for real-time deployment. Simran et al. (2020) explored DL architectures, including transformers, for detecting cyber threat indicators in Twitter streams, outperforming conventional ML methods but lacking integration with network-based features. These studies highlight transformers' potential but underscore the need for efficient variants like DistilBERT to balance performance and scalability.

Social networks' relational nature has prompted research into graph-based methods. Al Harbi et al. (2020) introduced XBully, a cyberbullying detection system that reformulates multimodal social media data as a heterogeneous network, using node embeddings to identify bullying behavior. Their approach effectively handled diverse data types (text, images) but was limited by the complexity of network construction. Ward et al. (2022) proposed a feature-driven ML model for cyberbullying detection on Twitter, incorporating tweet content, network information, and user characteristics, achieving an F1-score of 0.936. However, their reliance on predefined features restricted adaptability to evolving threats. Khairy et al. (2022) combined BoW with semantic latent features and offensive term lists, using an SVM classifier to achieve an F1-score of 0.78, but their text-only focus ignored network interactions.

Recent studies have explored hybrid approaches to address the limitations of single-modality models. Yuvaraj et al. (2021) integrated user-based, content-based, and role-based features with a deep decision tree classifier, improving cyberbullying detection performance over individual feature sets. Yazdinejad et al. (2023) proposed an ensemble DL model using LSTM and auto-encoders for cyber threat detection in industrial IoT environments, addressing imbalanced datasets but lacking social media-specific adaptations. Rios et al. (2019) utilized semantic analysis with fuzzy concept analysis and latent Dirichlet allocation to filter noise in social networks, enhancing influencer detection but not directly addressing cybercrime. Deliu et al. (2018) combined SVM and latent Dirichlet allocation to extract cyber threat intelligence from hacker forums, achieving 90% accuracy in identifying irrelevant content but requiring human intervention for validation. Table 1 presents a comprehensive comparison of related works on different key factors.

Table 1 Comparison of related works on key factors

References	Primary focus	Techniques used	Datasets used/data acquisition	Key findings
Balakrishnan et al. (2020)	Cyberbullying detection	J48, RF, NB	Twitter	Moderate accuracy; limited by small dataset and few labels
Nahar et al. (2014)	Semi-supervised cyberbullying	Fuzzy SVM	Streaming social media	Effective for dynamic data; struggles with small initial datasets
Dadvar and Eckert (2018)	Hate speech detection	LR, SVM	Wikipedia, Reddit, YouTube, Twitter	Feature selection improves accuracy; cross-platform challenges
Zhao and Mao (2017)	Cyberbullying detection	Auto-encoder, SVM	Social media	Robust representations; limited by SVM classification
Dewani et al. (2021)	Cyberbullying in Roman Urdu	SVM, Word2Vec	Social media	F1-score 0.78; lacks contextual embeddings
Iwendi et al. (2023)	DL for cyberbullying	Bi-LSTM, RNN, CNN	Social media	Bi-LSTM outperforms; requires tuning
Murshed et al. (2022)	RNN for cyberbullying	Modified RNN	Social media	Outperforms LSTM with class weighting
Dadvar and Eckert (2020)	BERT for cyberbullying	BERT	Social media	High accuracy; computationally intensive
Simran et al. (2020)	Cyber threat indicators	DL, transformers	Twitter	Outperforms ML; lacks network features
Prabowo and Azizah (2020)	Multimodal cyberbullying	Network embeddings	Social media	Handles diverse data; complex network construction
Graney-Ward, et al. (2022)	Feature-driven cyberbullying	ML, network features	Twitter	F1-score 0.936; limited adaptability
Bermeo et al. (2022)	Arabic cyberbullying	SVM, BoW, semantic features	Twitter	F1-score 0.78; text-only focus
Kim et al. (2021)	Multi-feature cyberbullying	Deep decision tree	Social media	Improved performance with hybrid features
Yazdinejad et al. (2023)	IoT threat detection	LSTM, auto-encoder	Industrial IoT	Addresses imbalance; not social media-specific
Rios et al. (2019)	Influencer detection	Fuzzy analysis, LDA	Plexilandia	Enhances efficiency; not cybercrime-focused
Deliu (2018)	Hacker forum intelligence	SVM, LDA	Nulled.IO	90% irrelevant content detection; needs human validation

3 Proposed cybercrime detection framework

The proposed framework, as shown in Fig. 1, integrates text preprocessing, feature extraction, and various intelligent learning models for cybercrime detection. It consists of six different components: Raw Data Input, Preprocessing Phase, Data Splitting, Feature Extraction, Modeling, and Evaluation Phase. The process begins with raw text which undergoes preprocessing steps such as tokenization, stop word removal, stemming, and lemmatization. Cleaned text is then passed through feature extraction methods like TF-IDF, Word2Vec, or FastText to convert it into numerical vectors.

These numerical vectors are then used as input features for the model training phase. In the case of traditional ML models like Naïve Bayes, Support Vector Machine, or Random Forest, the feature vectors are fed directly into the classifiers. For DL models such as LSTM, CNN, Bi-LSTM, or GRU, Word2Vec or FastText embeddings serve as the input layer to the network, allowing the model to learn complex patterns and contextual relationships in the text. While DistilBERT is used for the development of T-GNN model. In each case, the final output is a set of predicted labels, such as classifying a post as cyberbullying, terrorism-related, or a threat, thereby completing the cybercrime detection pipeline.

3.1 Data set collection

Perhaps one of the most pressing issues for society serves as the growing risk of extreme cybercrime. In solving this

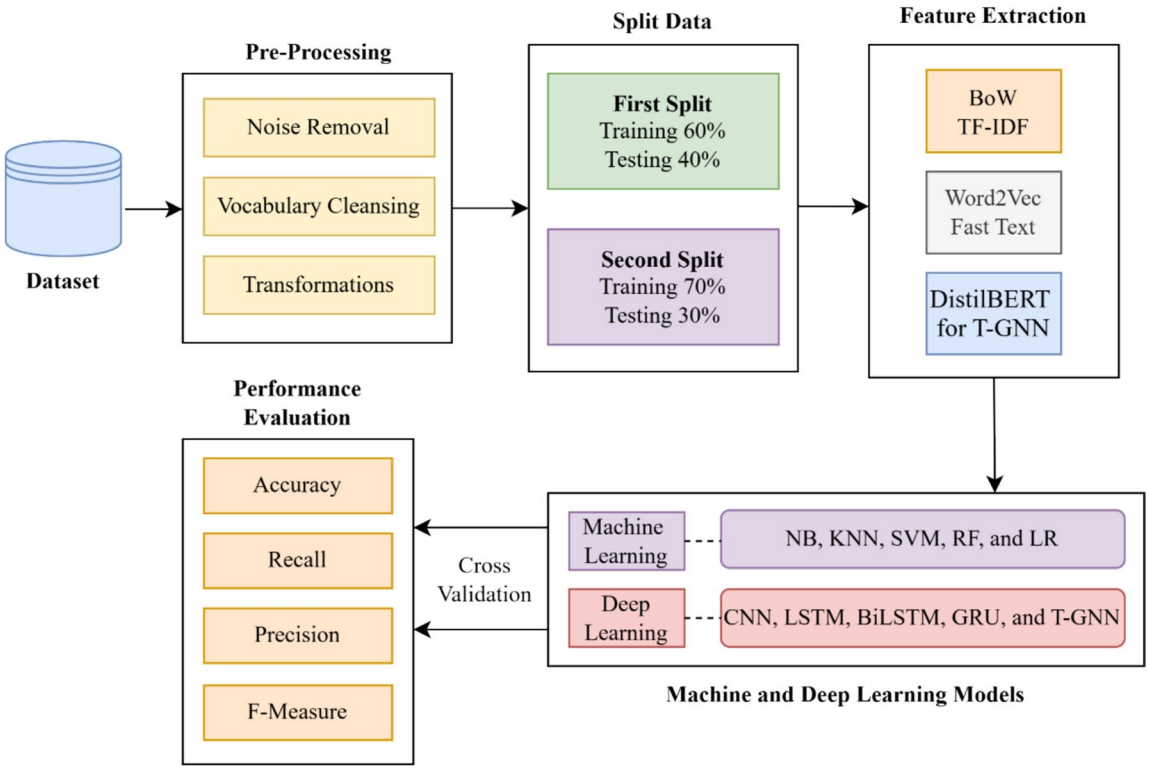


Fig. 1 Compact framework for cybercrime detection

problem, my research incorporates a dataset containing tweets which are assumed to be malicious and are situated on the published online platform known as Kaggle[1] with a total of sixty thousand and sixty-one tweets. Each tweet labeled as suspicious (0) can further be divided as cyberbullying, terrorism, and threat while the non-suspicious (1) label remains singleton.

3.2 Data preprocessing

One approach to address the task of social networks textual data preprocessing is set of comprehensive approaches to ML and DL. In this research, we implemented a detailed soft three-step process aimed at cleansing, detailing, and changing social media content into usable data with its first phase being as removal of irrelevant content that can hinder a model's performance, otherwise known as noise including but not limited to hashtags, URLs, emojis, special character, over punctuation, etc. (Zhou et al. 2025). The second step adds a language normalization level, which further improves data quality. This step usually involves formal language correction, phrase expansion, spelling check, and the removal of character repetitions often found in user-contributed content. The final phase transforms the cleaned text into a structured format suitable for computational processing. This includes tokenization (segmenting text into smaller units), removing stop words (frequent but semantically weak terms), converting all text to lowercase, and applying lemmatization to reduce words to their dictionary base forms (Logeswari et al. 2025). Together, these steps ensure the data is uniform, contextually coherent, and optimized for training intelligent models.

3.2.1 Stop words removal

Stop words are frequently occurring terms in natural language that, although essential for grammatical structure, offer minimal semantic value when analyzing text. Examples include function words such as "the," "and," "in," "on," "is," and "of." In this work, we systematically removed such words from the dataset to prevent them from skewing the model's understanding and to ensure that the algorithm focuses on more informative terms. This pruning process improves the signal-to-noise ratio in the input data, thereby enhancing downstream learning efficiency (Payette et al. 2025).

[1] https://www.kaggle.com/datasets/syedabbasraza/suspicious-tweets

3.2.2 Tokenization

Tokenization is a foundational step in text preprocessing, whereby textual input is decomposed into discrete units, or tokens, which may include words, numbers, or punctuation marks. This segmentation facilitates syntactic and semantic analysis by enabling the model to interpret the structure and meaning of the input text (Rai and Borah 2025). Our implementation emphasizes secure tokenization practices compliant with modern data protection standards, especially where sensitive or identifiable information may be involved. Moreover, the architecture allows for the ability to detokenize text, meaning the original text can be extracted from the tokens when necessary, thus, increasing the flexibility of the pipeline to not only complete certain analytical procedures in a permanent manner but also perform some tasks in a reversible manner.

3.2.3 Lemmatization

Lemmatization helps represent textual data in a more meaningful manner by employing context-sensitive linguistic rules to replace words with their base forms known as lemmas. Compared to stemming, which makes arbitrary cuts that leads to nonsensical forms, lemmatization uses several lexical resources and grammatical analysis to ensure the form being used is indeed valid and meaningful. As an example, “running” and “better” will be replaced by “good” and “run” respectively, depending on the context and grammatical role a word has in a sentence. This level of precision enhances model training generalization and semantic coherence (Islam et al. 2025).

3.3 Feature selection and extraction

Selection and extraction of features is monumental in transforming the unrefined social network text data into a structured format ready for ML models. In this stage, two popularly used methods TF-IDF and BoW are applied to the corpus to derive features that can add value (Khalid et al. 2014). The main goal is to obtain the most informative terms, in this case nouns, adjectives verbs because they tell a lot about context, meaning especially in cybercrime detection. While these content words are central, function words including pronouns and Part-of-Speech (POS) tags may also carry syntactic and contextual significance and are thus not entirely disregarded.

As emphasized in Lampos et al. (2017), selecting the most relevant features is crucial for enhancing the learning capacity and generalization of predictive models. In this work, we adopt a structured approach to identify features that are most indicative of cybercrime-related expressions. The selected features are then utilized to train and evaluate ML classifiers, ensuring that the models learn patterns grounded in semantically relevant indicators (Corizzo et al. 2020).

3.3.1 Term frequency–inverse document frequency

A statistical method known TF-IDF analyzes a word’s relevance based on a specific document within a set of documents. It collects two major metrics: Term Frequency or TF, which calculates the frequency of a term within a given document, and Inverse Document Frequency or IDF, which assesses how common or rare the term is relative to the whole set of documents.

The intuition behind TF-IDF is that words which occur frequently in a specific document but infrequently across all documents are likely to carry more discriminative information. Hence, such terms receive a higher TF-IDF score, making them valuable for tasks like classification or clustering. Conversely, highly frequent words that appear in most documents (e.g., "the," "and") are assigned lower scores due to their limited informativeness. The TF-IDF score for a term t in document d from a corpus D is computed using the following formula:

$$TF - IDF(t, d, D) = TF(t, d) \times \log\left(\frac{N}{|\{d \in D : t \in d\}|}\right) \tag{1}$$

where, $TF(t, d)$ is the frequency of term t in document d, N is the total number of documents in the corpus, $|\{d \in D : t \in d\}|$ is the number of documents in which the term t appears.

The importance of each feature in the weighting schema also draws attention to the most distinctive words within a document to improve text classifier performance.

3.3.2 Bag-of-words

The Bag-of-Words model, or BoW, is regarded as one of the first approaches in the field of transforming unstructured texts into a structured numerical format applicable by ML algorithms. This model represents documents based on word existence without tracking word order or semantics. For BoW representation to be calculated, a vocabulary comprising of all unique words in the whole corpus is established first. After that, every document is turned into a vector, where every position is associated with a word from the vocabulary.

In this form, a binary value (usually 1 or 0) reflects whether the word is contained within the document or not. This creates a sparse matrix because a large proportion of the values are zero. Even though this is a relatively basic approach, it does serve as a useful foundation for grasping the text structure and is popular among older ML

frameworks. The resulting matrix is constructed in accordance with the feature representation required for subsequent training and evaluation of the model (Wu et al. 2025).

3.4 Word embedding

Unlike the sparse representation models like BoW, word embedding retains the syntactic and semantic relations of the words, thus offering a more effective approach. It is now understood that words with context meaning are situated closer to one another in the vector space. By capturing essence meaning in the relationships between words, this method significantly shrinks the dimensionality of the data while improving the efficiency and accuracy of the learning algorithms. A DL system utilizes numerical inputs, relying on dense vectors containing words instead of text, thus enabling efficient processing of language data. In this work, two popular cybercrime detection frameworks are facilitated with DL architectures: Word2Vec and FastText, which are known for efficient word embedding techniques (Singh and Sharma 2025).

3.4.1 Word2Vec

Word2Vec is a neural embedding technique that predicts the context relations among words to learn their vector representations. It comprises the Continuous Bag-of-Words (CBOW) and Skip-gram models. In this study, we utilize the Skip-gram model, which aims to predict surrounding context words given a central word. The model is trained by maximizing the probability of observing context words within a given window size around a target word. The training objective is represented as:

$$argmax_{\theta} \prod_{c \in C} p(c|w;\theta) \tag{2}$$

As previously described, C is a set of context words, with w being a target word and C denotes some target word's context. In this case, θ stands for a parameter of the embedding model. The learning procedure remains the same – it updates weights connected to the first row of the weight matrix for both input and hidden layer. For example, the vector representation of a word w_i can be denoted as:

$$h = W_I^T c = W_k^T = v_{w_i}^T \tag{3}$$

The learned vectors enable the model to capture semantic similarities, such that words like "bigger" and "large" are located near each other in the vector space. Figure 2 demonstrates both the CBOW and Skip-gram architectures, illustrating how contextual information flows through the model layers.

3.4.2 FastText

Traditional word embeddings assign a single vector to each word, making them ineffective in dealing with rare or unseen words. FastText addresses this limitation by incorporating subword information. Rather than treating words as atomic units, FastText decomposes them into character-level n-grams and learns embeddings for these subword units.

For example, if the word "introduction" is processed using trigrams (n = 3), it would be represented by n-grams such as: < int, tro, rod, odu, duc, uct, cti, tio, ion > . This allows FastText to generalize to new words by leveraging shared subword components. By way of example, the n-grams "int"

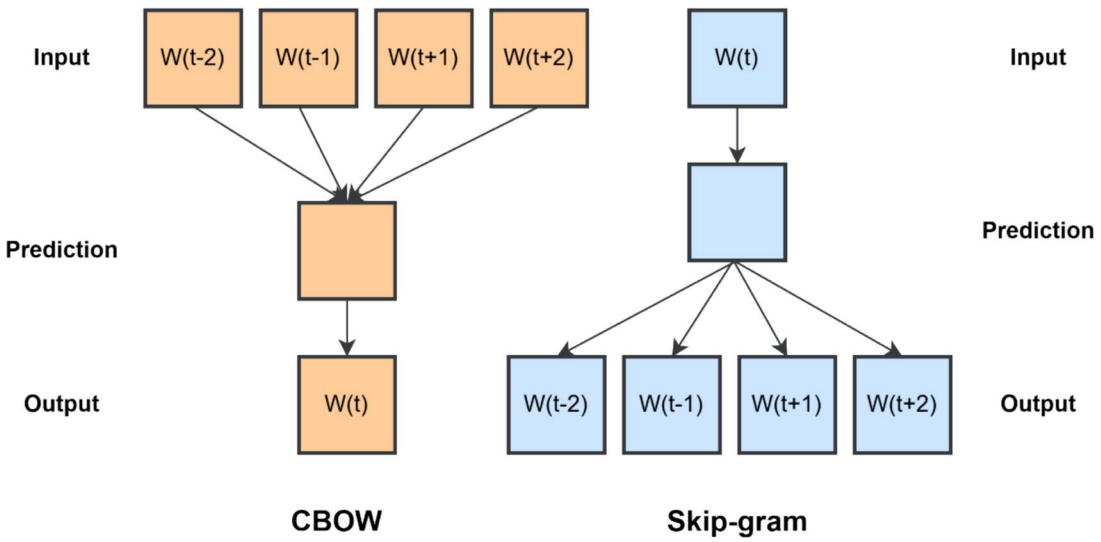


Fig. 2 Continuous bag-of-words and skip-gram architecture

and “duc” of the word "introduces" independently contribute towards forming an accurate vector representation that captures the essence of the term utilizing learned patterns. The scoring function of FastText which computes similarity of the word w and context C is as follows:

$$S(w,c) = \sum_{g \in g_w} Z_g^T v_c \tag{4}$$

where, g_w is the set of character n-grams for word w, Z_g is the vector representation of the n-gram g, v_c is the context vector.

As previously discussed, offering a powerful substitute to embedding approaches (Haydar and Dhannoon 2025), this method increases FastsText’s effectiveness in handling out-of-vocabulary terms as well as modeling morphology.

3.5 Intelligent learning models

In the below subsections, we discuss different types of ML and DL models utilized in this study.

3.5.1 Machine learning models

In this study, we implemented a total of five different ML models aimed at solving classification problems. Each of those five models that gained considerable popularity as classifier models has been described in a matter of notation with equations below and explanation of terminology used that follows afterwards:

3.5.1.1 Naïve bayes Naive bayes is a probabilistic classifier applying Bayes’ Theorem and is based on classifying incoming data one feature at a time, employing conditional independence as the relation among features. Thus, it formulates does make some assumptions that allow for calculation simplifications in high dimensional data like text classification which is why it is the most efficient.

$$P\left(C_k|x\right) = \frac{P\left(x|C_k\right) \bullet P(C_k)}{P(x)} \tag{5}$$

where, $P\left(C_k|x\right)$ is the posterior probability of class C_k given input x, $P\left(x|C_k\right)$ is the likelihood of input x given class C_k, $P(C_k)$ is the prior probability of class C_k, $P(x)$ is the probability of input x (acts as a normalization factor). The prediction is made by:

$$\hat{y} = \arg\max_{C_k} P\left(C_k x\right) \tag{6}$$

3.5.1.2 K-nearest neighbor K-nearest neighbor is a non-parametric, instance-based learning algorithm. It classifies a data point based on the majority class among its kk nearest neighbors in the feature space.

$$d\left(x, x_i\right) = \sqrt{\sum_{j=1}^{n} \left(x_j - x_{ij}\right)^2} \tag{7}$$

where, $d\left(x, x_i\right)$ is the Euclidean distance between the query point x and a training instance x_i, x_j, x_{ij}: j-th features of query and training point, n is the number of features. The predicted class is the majority vote among the k nearest neighbors.

3.5.1.3 Support vector machine Support vector machine is a powerful linear (and non-linear via kernel tricks) classifier that finds the hyperplane that best separates data into classes with the maximum margin.

$$f(x) = wTx + bf(x) = w^T x + b \tag{8}$$

Prediction is performed using:

$$\hat{y} = \text{sign}(f(x)) \tag{9}$$

where, w is the weight vector orthogonal to the hyperplane, x is the feature vector, b is the bias term, and sign () is the sign function determines the side of the hyperplane. The optimal hyperplane maximizes the margin:

$$\max \frac{2}{|w|} \tag{10}$$

3.5.1.4 Random forest classifier Random Forest is an ensemble learning method that builds multiple decision trees and merges their predictions. It improves accuracy and controls overfitting.

$$\hat{y} = \text{mode}\left(T_1(x), T_2(x), \ldots, T_m(x)\right) \tag{11}$$

where, $T_i(x)$ is the prediction from the i-th decision tree, m is the number of trees in the forest, mode is the most frequent predicted class among all trees. Each tree is trained on a bootstrap sample and considers a random subset of features.

3.5.1.5 Logistic regression Logistic Regression is a linear model used for binary classification. It models the probability that an instance belongs to a particular class using the sigmoid function.

$$P(y = 1x) = \sigma\left(w^T x + b\right) = \frac{1}{1 + e^{-\left(w^T x + b\right)}} \tag{12}$$

where, $P(y = 1|x)$ is the probability that the class label is 1 given input x, w is the Weight vector, b is the bias term, and σ() is the Sigmoid function. The model assigns class 1 if $P(y = 1|x) \geq 0.5$, otherwise class 0.

3.5.2 Deep learning models

Deep learning models have brought significant advancements across various domains, especially in NLP. This study capitalizes on the strength of DL by examining four widely adopted architectures for text classification tasks. Each model is tailored to capture distinct linguistic features and contextual dependencies within the textual data.

3.5.2.1 Convolutional neural networks Originally popularized in image recognition, CNNs have also demonstrated strong performance in NLP tasks. In the context of text classification, CNNs function by applying filters over the word embeddings, detecting n-gram patterns and semantic features. These filters slide across the input text to capture spatial hierarchies of information, and the resulting feature maps are passed through pooling layers to downsample and retain the most critical features.

Max pooling, a common downsampling technique, is employed to reduce dimensionality by selecting the most prominent feature within each window. The pooled outputs are then flattened and fed into dense layers, culminating in the final classification layer. However, a major challenge in text applications is the high input dimensionality caused by the large vocabulary size. While image data typically have a fixed number of channels (e.g., RGB = 3), text data may involve thousands of unique tokens, making CNNs computationally intensive for such tasks.

3.5.2.2 Long short-term memory To overcome limitations in modeling long-term dependencies, LSTM networks are employed. Unlike traditional RNNs, LSTMs are equipped with gating mechanisms that regulate the flow of information. This enables them to excel at detecting patterns within sequential data, for instance, language. The architecture has an input gate, output gate, and forget gate. These gates improve the model's ability to preserve relevant information while discarding noise throughout time steps, increasing noise robustness on long sequences.

$$\text{Input}_t = \sigma\left(W_i\left[h_{t-1}, x_t\right] + b_i\right) \tag{13a}$$

$$\text{Output}_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right) \tag{13b}$$

$$\text{Forget}_t = \sigma\left(W_f\left[h_{t-1}, x_t\right] + b_f\right) \tag{13c}$$

Here, x_t represents the input at time t, $\mathrm{h}_{\mathrm{t}-1}$ is the previous hidden state, W and b are the respective weight matrices and biases, and σ is the sigmoid activation function.

3.5.2.3 Bidirectional LSTM To further increase context awareness, Bi-LSTM networks are applied. Unlike LSTMs with a single directional input to the sequence, Bi-LSTMs read the input backward and forward. This allows the model to develop a more complete understanding of contextual dependencies, which is vital for sentiment classification and intent detection.

Mathematically, the output at time t is represented as:

$$Y_t = \left[\overrightarrow{h_t}, \overleftarrow{h_t}\right] \tag{14}$$

where $\overrightarrow{h_t}$ and $\overleftarrow{h_t}$ denote the forward and backward hidden states, respectively, and the bracketed term indicates vector concatenation.

3.5.2.4 Gated recurrent unit Gated Recurrent Unit is a simplified variant of LSTM networks, created for sequential tasks with less parameters and quicker convergence time. GRUs strip away the forget gate and input gate, merging them into a single update gate, while a reset gate dictates the extent of past data to be forgotten. Their simpler structure enables faster training while still capturing essential sequence dependencies.

The update and reset gates are calculated as:

$$z_t = \sigma\left(W_z x_t + U_z h_{t-1} + b_z\right) \tag{15a}$$

$$r_t = \sigma\left(W_r x_t + U_r h_{t-1} + b_r\right) \tag{15b}$$

The hidden state h_t is updated using:

$$h_t = \left(1 - z_t\right) \odot h_{t-1} + z_t \odot \widetilde{h_t} \tag{16}$$

where $\widetilde{h_t} = \tanh\left(W_h x_t + U_h\left(r_t \odot h_{t-1}\right)\right)$, and ⊙ denotes element-wise multiplication.

Each structure stands out, as CNNs process nearby patterns, Bi-LSTM allows in-depth access to context and GRU results in efficient handling. With all these techniques, it is possible to construct robust systems for detecting cybercrimes using text.

3.5.2.5 Transformer-augmented graph neural network The framework makes use of DistilBERT for understanding the text and GAT for representing connections between nodes. Figure 3 below explains the design of T-GNN and below we break down each component of the architecture.

- Text Encoder (DistilBERT): Encodes tweet text into 768-dimensional embeddings, capturing contextual semantics. DistilBERT is chosen for its efficiency compared to BERT.
- Graph Encoder (GAT): Models network interactions using attention mechanisms to weigh neighbor contributions. Each node aggregates features from its neighbors, weighted by attention scores.

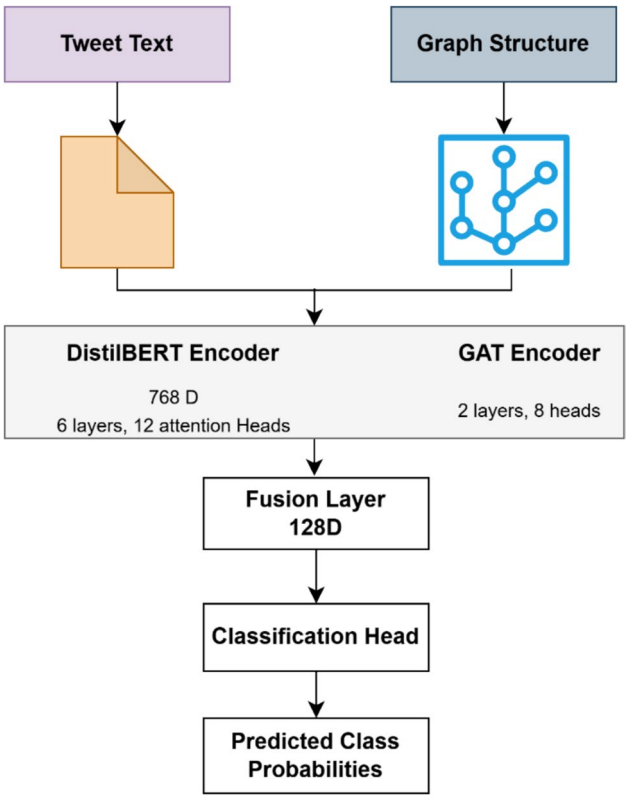


Fig. 3 Transformer-augmented graph neural network architecture

- Fusion Layer: Concatenates DistilBERT and GAT outputs, followed by a fully connected layer with ReLU activation.
- Classification Head: A softmax layer outputs probabilities for suspicious/non-suspicious or multi-class labels.

DistilBERT embedding: $e_t = \text{DistilBERT}(t)$, where t is the input tweet.

GAT update for node i:

$$h_i^{(l+1)} = \sigma\left(\sum_{j\in N_i} \alpha_{ij} W^{(l)} h_j^{(l)}\right), \qquad \alpha_{ij} = soft\max\left(Leaky\text{ReLU}\left(a^{\mathrm{T}}\left[Wh_i \mid Wh_j\right]\right)\right) \tag{17}$$

where, $h_i^{(l)}$ is the node feature at layer l, N_i is the neighbor set, α_{ij} is the attention coefficient, and (W, a) are learnable parameters. Fusion: $z = [e_t || h_i]$, followed by $y = \text{softmax}\left(W_z z + b\right)$.

4 Experimental settings

To assess the effectiveness of various ML and DL models for cybercrime detection in social media contexts, we conducted extensive experiments using a real-world dataset comprising Twitter (X) posts. The evaluation focused on standard classification metrics: Accuracy, Precision, Recall, and F1-score, ensuring a robust analysis of model performance.

The preprocessing pipeline consisted of systematic text normalization steps to enhance data quality and reduce noise. This included tokenization using the RegexTokenizer, stemming via the PorterStemmer, and lemmatization using WordNetLemmatizer. Stop words, special characters, URLs, and emojis were also removed. All text was transformed to lowercase, and duplicate characters were normalized. Word embeddings were configured with a 300-dimensional vector space to capture semantic richness in textual data. For the Skip-gram-based Word2Vec, it was critical to fine-tune the embedding parameters to capture relevant semantic relationships effectively. During hyperparameter tuning, we experimented with several window sizes in the range of Teepapal (2025); Tarade and Das 2025; Nahar et al. 2014; Zhao and Mao 2017). The optimal performance was achieved at a window size of 5, which provided a balanced context around the target word, enough to capture surrounding semantics in short tweets without introducing too much noise. Moreover, we tested learning rates in the range [0.005, 0.01, 0.025], which are commonly used for Skip-gram training. We observed that a learning rate of 0.01 consistently produced better convergence and model stability during training. These settings were selected based on manual hyperparameter tuning and validated through repeated cross-validation and downstream performance evaluation using our deep learning classifiers.

For feature extraction and representation, both TF-IDF and word embedding methods (Word2Vec and FastText) were employed. These features were used to train traditional ML models such as Naïve Bayes, LR, SVM, RF, and KNN.

In the case of LR, feature scaling was performed prior to training, and the model was validated using classification reports, ROC-AUC scores, and confusion matrices. The Random Forest classifier was configured with 500 decision trees (n_estimators = 500), using the Gini index as the splitting criterion. The dataset was partitioned using an 80–20 train-test split, and the model was trained and evaluated accordingly.

The DL models explored include CNN, LSTM, Bi-LSTM, and GRU. In particular, the Bi-LSTM network featured two sequential recurrent layers with 200 and 400 neurons, respectively, followed by three fully connected (dense) layers comprising 128, 64, and 32 neurons. To mitigate overfitting, three dropout layers were incorporated between dense layers. A major contribution of this study is the development of a T-GNN model, designed to integrate textual and topological information for superior classification. The architecture of the T-GNN includes:

- Textual Encoding: A fine-tuned DistilBERT model that transforms raw tweet text into 768-dimensional contextual embeddings using a 6-layer, 12-head transformer architecture.
- Graph Encoding: A GAT with 2 layers and 8 attention heads, applied to a graph structure where nodes represent tweets or users and edges represent retweets, mentions, or interactions. Graph-level features included TF-IDF scores and sentiment polarity values.
- Fusion Layer: The outputs from DistilBERT and GAT encoders are concatenated and passed through a 128-unit fully connected layer activated by ReLU. A dropout rate of 0.3 was applied to prevent overfitting.
- Classification Head: The fusion embedding is passed into a softmax layer that outputs class probabilities, either for binary classification (suspicious vs. non-suspicious) or for multiclass classification (e.g., cyberbullying, terrorism, threats).

The models were implemented in Python using libraries such as Scikit-learn, TensorFlow, Keras, NumPy, and NLTK. Experiments were conducted on a Windows 10 platform with Intel Core i5 CPU and 16 GB RAM. The dataset was pre-processed as described, with embedding dimensions set to 300 for Word2Vec/FastText and 768 for DistilBERT. We employed 5-fold cross-validation and experimented with 60–40 and 70–30 train-test splits to ensure the statistical reliability of the reported metrics. The T-GNN is trained end-to-end using cross-entropy loss, with AdamW optimizer and a learning rate of 2e-5. Hyperparameters were tuned manually, including learning rates (2e-5 for T-GNN, 0.01 for ML), batch sizes (16 for T-GNN, 32 for others), and epochs (10 for T-GNN, 50 for ML).

This integrated evaluation of ML, DL, and GNN-based models provides a comprehensive understanding of their respective strengths and effectiveness in detecting cybercrime on online social networks. Through rigorous experimentation and fine-tuned architectures, our framework demonstrates significant potential for real-world deployment in digital safety and reliability contexts.

4.1 Evaluation metrics

To assess the performance of the proposed classification models, we employ a set of standard evaluation metrics that are widely used in machine learning and statistical modeling. These metrics provide a comprehensive view of how well the models perform, particularly in distinguishing between different categories within the data.

- Accuracy: Accuracy is a general indicator of how frequently the model makes correct predictions. Here, it is described as the ratio of the number of samples that were correctly classified, both positive and negative, to the total number of samples assessed. This metric offers a quick overview of performance, especially when the dataset is balanced.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{18}$$

- Precision: Precision focuses on the quality of positive predictions. It quantifies the proportion of true positive cases among all instances the model predicted as positive. Precision is particularly important in scenarios where the cost of false positives is high.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{19}$$

- Recall: Also referred to as sensitivity or the true positive rate, recall captures the model's ability to detect all relevant positive instances. It measures how effectively the model identifies actual positives in the dataset, which is vital in applications where missing a positive case could be critical.

$$\text{Recall} = \frac{TP}{TP + FN} \tag{20}$$

- F1-Score: A solitary measure can be used to determine both the precision and recall with the help of F1 score, or F-measure. This gauge is particularly effective when the two groups are imbalanced or when both type I and type II errors value greatly. It can be obtained as the harmonic mean of precision and recall which guarantees that if one of the two metrics yields low score, the final score will be considerably low.

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{21}$$

These evaluation metrics together offer a well-rounded framework for understanding the classification performance of our models, allowing us to compare their effectiveness across different experimental conditions.

5 Experimental results

To assess the performance of our proposed framework for cybercrime detection, we conducted a series of experiments focusing on three major categories: cyberbullying, terrorism, and threats. The evaluation is divided into two parts. The first part highlights the outcomes of traditional ML models, and the second part analyzes the results obtained using DL architectures.

5.1 Cybercrime detection using machine learning models

Table 2 provides a detailed comparison of five prominent ML classifiers: NB, SVM, KNN, RF, and LR. Each classifier was tested using three distinct feature extraction strategies: Count Vectorization, TF-IDF, and a hybrid TF-IDF method incorporating both unigrams and bigrams. The experiments were performed using two data splits: 60–40% and 70–30% for training and testing. The corresponding graphical results are illustrated in Figs. 4 and 5, respectively.

In the 60–40% split, RF consistently emerged as the top-performing model, attaining the highest accuracy of 91.85% with TF-IDF (unigrams + bigrams). RF with TF-IDF and KNN with TF-IDF (unigrams + bigrams) followed closely, reaching an accuracy of 91.63% and 91.61%, respectively. Interestingly, RF achieved the best F1-score of 91.65% using both TF-IDF and TF-IDF with unigrams and bigrams. Both LR and NB outperformed SVM, which showed relatively weaker results across most of the parameters. Despite this, RF maintained a commendable 90.40% accuracy using Count Vectorization.

The trends remained largely consistent in the 70–30% split. RF once again demonstrated superior accuracy, peaking at 91.98% with TF-IDF (unigrams + bigrams). LR also showed strong results, achieving 90.12% accuracy, while RF secured an F1-score of 91.79% with the same feature representation. Across both splits, TF-IDF (particularly when combined with unigrams and bigrams) consistently outperformed Count Vectorization, affirming its effectiveness in feature representation for ML-based cybercrime detection.

These findings highlight that ensemble-based models like RF, when paired with rich text representations, offer

Table 2 Comparison of ML algorithms with different feature extraction techniques across evaluation metrics and data splits

Split	Feature extraction	Machine learning algorithms	Accuracy	Recall	Precision	F-measure
First split (60–40%)	*Count Vectorization*	NB	86.70	90.34	88.20	88.71
		KNN	73.01	77.68	76.46	74.42
		SVM	82.11	81.30	80.00	80.24
		RF	90.40	90.71	90.11	91.00
		LR	87.64	87.32	88.27	87.56
	TF-IDF	NB	89.71	90.32	89.21	89.46
		KNN	74.00	77.62	77.41	74.00
		SVM	82.37	81.33	81.64	82.27
		RF	91.63	90.73	90.79	**91.65**
		LR	89.72	90.35	89.21	89.61
	TF-IDF + Unigrams/Bigrams	NB	89.72	90.23	89.18	89.71
		KNN	91.61	80.66	79.46	79.00
		SVM	83.10	81.94	82.91	82.38
		RF	**91.85**	**90.87**	**90.95**	**91.65**
		LR	90.12	90.00	89.82	89.92
Second split (70–30%)	*Count Vectorization*	NB	86.71	90.34	88.22	88.72
		KNN	73.11	77.66	76.46	74.45
		SVM	82.12	81.37	80.87	80.31
		RF	90.43	90.77	90.17	91.14
		LR	87.66	87.35	88.20	87.79
	TF-IDF	NB	89.72	90.34	89.22	89.72
		KNN	74.01	77.66	77.46	74.00
		SVM	82.38	81.37	81.91	82.38
		RF	91.66	90.77	90.88	91.66
		LR	89.72	90.35	89.22	89.72
	TF-IDF + Unigrams/Bigrams	NB	89.72	90.34	89.22	89.72
		KNN	76.01	80.66	79.46	79.00
		SVM	83.11	82.33	82.91	82.38
		RF	**91.98**	**90.98**	**90.93**	**91.79**
		LR	90.12	90.00	89.97	89.98

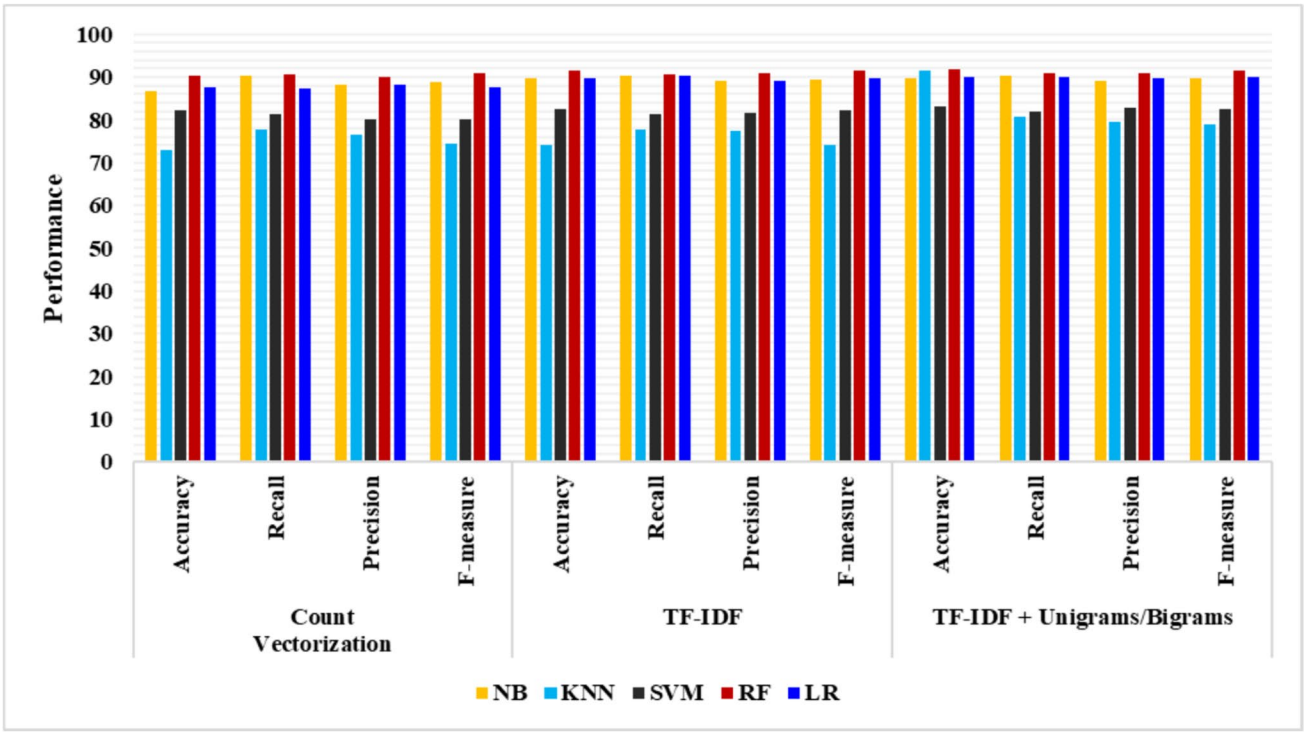


Fig. 4 Comparison of machine learning algorithms on first split (60–40%) using various feature extraction techniques and evaluation metrics

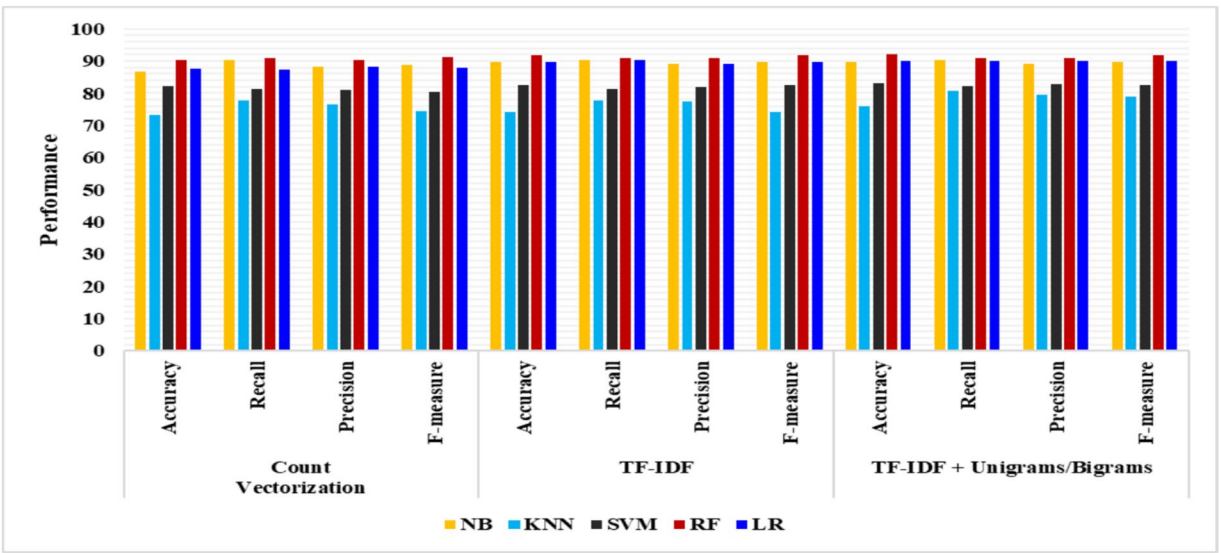


Fig. 5 Comparison of machine learning algorithms on second split (70–30%) using various feature extraction techniques and evaluation metrics

significant advantages for classifying harmful content on social networks.

5.2 Cybercrime detection using deep learning models

Table 3 presents the performance comparison of various deep learning models CNN, LSTM, Bi-LSTM, GRU, and T-GNN across two experimental settings (60–40 and 70–30 splits), using three types of feature extraction techniques:

Word2Vec, FastText, and DistilBERT combined with sentiment scores. Evaluation is performed using four standard metrics: Accuracy, Precision, Recall, and F1-score, since relying solely on accuracy can be misleading in the presence of imbalanced data, which is often the case in real-world social media datasets. The corresponding graphical results are presented in Fig. 6.

Among the traditional deep learning models, Bi-LSTM consistently emerges as a top performer, achieving accuracy scores of 97.01% and 97.76% in the first and second splits respectively using Word2Vec embeddings. FastText embeddings also proved to be highly effective with Bi-LSTM, with performance figures closely following 97.20% and 97.18%. This model also achieves high F1-scores, indicating a good balance between precision and recall, making it reliable for capturing both the presence and relevance of cybercrime indicators in text.

The LSTM model, another recurrent architecture, also shows strong results, particularly when paired with FastText (96.14% and 96.00% accuracy). CNN, though generally effective in capturing spatial dependencies in text, shows slightly lower performance compared to RNN variants, achieving accuracies around 94.16% using Word2Vec. Across all configurations, GRU underperforms slightly when compared to Bi-LSTM and LSTM, suggesting that its simpler architecture may not fully capture the contextual depth required for this task.

The highlight of the study, however, is the performance of the T-GNN, which surpasses all baseline models. T-GNN leverages the contextual strength of DistilBERT for textual encoding and the structural advantages of GAT for modeling interactions between users and content. With this architecture, the model achieved an outstanding accuracy of 98.15% on the 60–40 split and 98.42% on the 70–30 split, along with superior precision, recall, and F1-scores across both settings.

In addition to raw accuracy, T-GNN benefits significantly from its graph-based feature integration, which enhances the detection of coordinated or bot-driven attacks often missed by sequential models. The inclusion of sentiment scores further improves recall, especially for emotionally charged content like cyberbullying or threats. Moreover, the use of DistilBERT, a lightweight alternative to BERT, offers a scalable solution suitable for deployment in real-time systems, without compromising performance.

In summary, while conventional DL models like Bi-LSTM and LSTM deliver robust performance, the T-GNN model clearly outperforms all others in both experimental splits, validating the effectiveness of combining transformer-based contextual embeddings with graph neural structures for cybercrime detection on social media platforms.

5.3 Results discussion

In our baseline experiments defined within the proposed cybercrime detection framework, DL techniques significantly outperform traditional ML techniques, demonstrating the efficacy of the algorithms. With each DL model of different experimental setups and evaluation criteria, there

Table 3 Comparison of deep learning algorithms using different feature extraction techniques across evaluation metrics and data splits

Split	Feature extraction	Deep learning algorithms	Accuracy	Recall	Precision	F-measure
First split (60–40%)	*Word2Vec*	CNN	94.16	93.12	92.65	92.15
		LSTM	96.00	95.01	94.54	95.00
		Bi-LSTM	97.01	94.10	94.18	94.00
		GRU	93.90	93.65	92.61	92.99
	Fast text	CNN	95.00	95.00	94.44	95.09
		LSTM	96.14	94.25	94.11	94.01
		Bi-LSTM	97.20	93.13	92.61	93.00
		GRU	95.19	95.09	95.00	95.10
	DistilBERT + Sentiment	**T-GNN**	**98.15**	**97.82**	**97.95**	**97.88**
Second split (70–30%)	*Word2Vec*	CNN	95.00	94.81	95.00	95.02
		LSTM	96.10	92.67	92.65	93.29
		Bi-LSTM	**97.76**	95.62	95.00	95.50
		GRU	95.07	94.93	95.00	95.13
	Fast text	CNN	94.16	93.12	92.65	92.15
		LSTM	96.00	95.01	94.54	95.00
		Bi-LSTM	97.18	94.10	94.18	94.00
		GRU	93.90	93.65	92.61	92.99
	DistilBERT + Sentiment	**T-GNN**	**98.42**	**98.10**	**98.25**	**98.17**

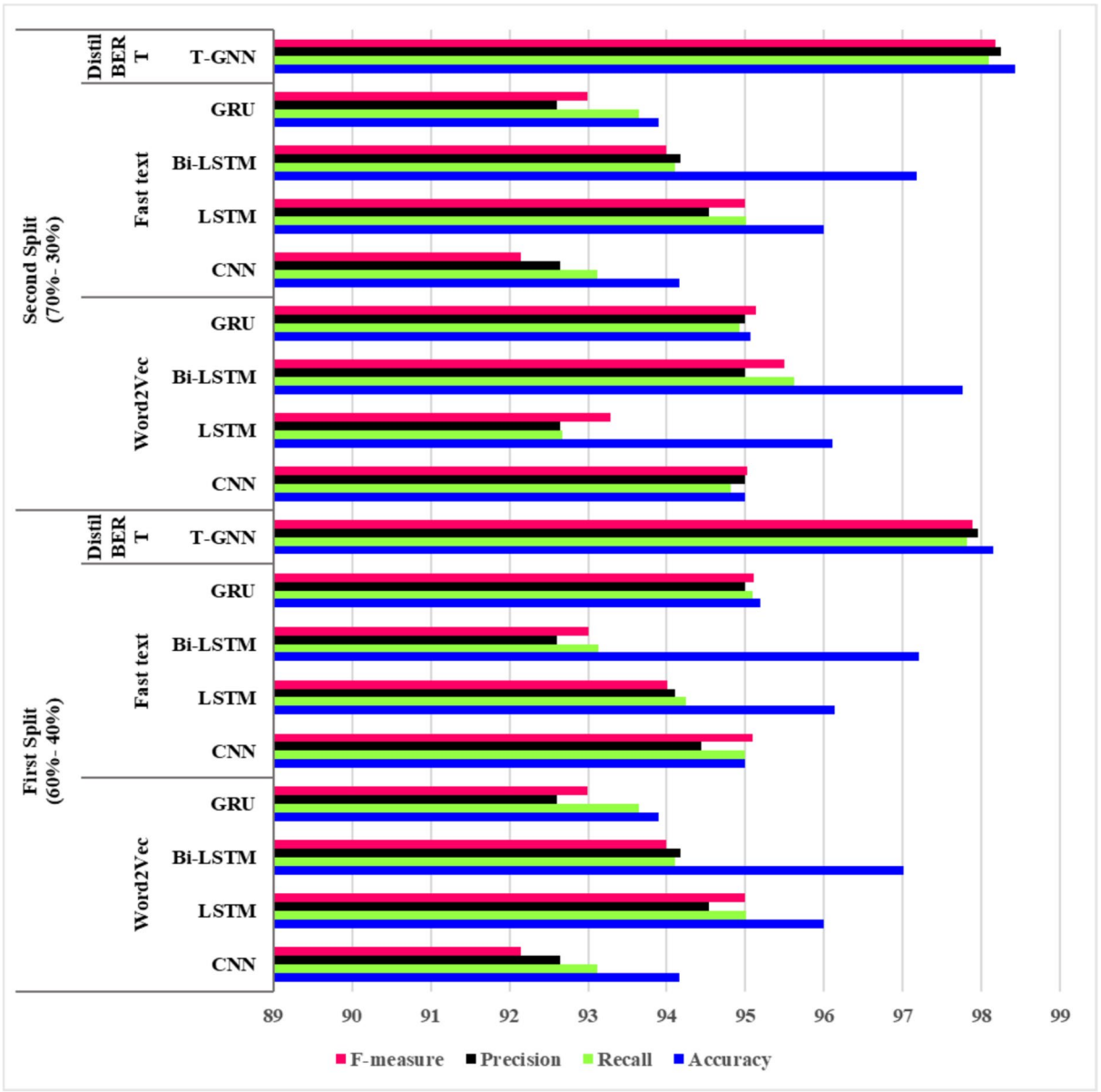


Fig. 6 Performance comparison of deep learning algorithms with different feature extraction methods across evaluation metrics and splits

was always an increase in accuracy, precision, recall, and F1-score, proving the ability of DL models to capture complex patterns in text data which is unstructured and intricate.

In the ML category, RF achieved the best performance, with an accuracy of 91.85% in the 60–40 split using TF-IDF features with unigrams and bigrams. This improved marginally to 91.98% in the 70–30 split, reaffirming RF's stability and effectiveness in traditional feature-based classification tasks. While other models like KNN and NB also showed reasonable results, their performance fell short when compared to the deep learning alternatives.

Among the standard DL models, Bi-LSTM emerged as a clear front-runner, achieving up to 97.76% accuracy in the second split with Word2Vec embeddings and consistently outperforming CNN, LSTM, and GRU across both splits and embedding methods. Notably, FastText embeddings slightly improved Bi-LSTM performance in several instances, supporting its effectiveness in capturing subword information and morphological patterns.

However, the introduction of the T-GNN redefined the performance benchmark. By integrating DistilBERT for text embeddings and GAT for modeling relational data such as user interactions, T-GNN effectively captured both semantic and structural patterns. The architecture produced these impressive results with consistent performance metrics sentiment score integration. 98.15 in the first split and

98.42 recall boost for emotionally charged content detection enhanced scores through boosting boosts lower bob style deadlines. Moreover, graph-based features enabled detection of cyberbullying coordinated bob-style attacks often missed by sequential models.

T GNN's efficiency and accuracy are distinguishing features that set it apart from the rest. Operational scalability is guaranteed by employing the transformer, DistilBERT, which is both lightweight and powerful in its computation, thus making it ideal for real-time application in cybercrime monitoring systems. T GNN's achievement emphasized the need to fuse contextual word representations and structure analysis in social media settings.

Our analysis revealed important insights:

1. The addition of Bi-LSTM turned out to be beneficial as it is the best performing DL model out of the traditional ones, especially when used with FastText or Word-2Vec due to the retention of context from previous and upcoming tokens resulting in increased performance.
2. The inclusion of transformer-based text encoding and graph-based structural features improved performance immensely. This is evident in T GNN which was able to outperform all other baseline models achieving the highest scores across all evaluation metrics and splits.
3. Among ML models, RF consistently performed best, yet failed to match the predictive power of deep neural networks.
4. Sentiment scores contribute positively to recall, particularly for emotionally sensitive cybercrime categories like harassment or threats.
5. Graph features introduced through GAT modules enhanced detection accuracy for cybercrime patterns that involve user interaction networks, such as retweets or replies in coordinated campaigns.

In conclusion, our results clearly demonstrate the superiority of advanced deep learning models, especially T-GNN, in tackling the growing challenge of cybercrime detection on social networks. The fusion of text semantics, sentiment, and graph-based user interaction features marks a promising direction for future research and deployment in intelligent content moderation systems.

6 Conclusion

The rise of the online world has unfortunately led to a significant increase in cybercrime, posing serious risks to individuals and society. This study addresses the critical challenge of cyberbullying detection, a key aspect of ensuring digital safety, by proposing a comprehensive cybercrime detection framework. The model evaluates nine classification techniques, including deep learning and traditional machine learning methods, across multiple feature extraction strategies. Of all the evaluated models, the T-GNN was the best with their provided results achieving an accuracy of 98.42 under the 70–30 data partition setting. This shows that the T-GNN architecture brings together the semantic strength of DistilBERT with the relational learning of GAT, making it capable of acquiring comprehension of both text and network cyberspace behavior. Given that DistilBERT represents a lightweight implementation of a transformer model when compared to BERT, its inclusion in T-GNN's structure facilitates scalable implementation, hence, making T-GNN feasible for real-time detection of cybercrimes in operational systems. In addition, the model's stability regarding the presence of diverse features like sentiment and network features demonstrates its ability to endure complex online settings.

For future work, we intend to examine how edge and cloud-based platforms might be used for deployment to improve the speed of threat detection on social media. Moreover, responsible AI use to detect cybercrime necessitates addressing ethical issues like data privacy, model bias, and lack of transparency which cannot be neglected.

Author contributions [Mohammed Wasid and Abdullah Yahya Abdullah Amar] Conceptualization, Methodology, Software, Investigation, Data curation, Writing—original draft. [Shahnawaz Ahmad and Jiyaul Mustafa] Visualization, Validation, Formal analysis, Writing—review & editing.

Funding Not Applicable.

Data availability The dataset is available from the corresponding author on reasonable request.

Declarations

Conflict of interest On behalf of all authors, the corresponding author states that there is no conflict of interest.

Research involving human and /or animals Not applicable.

Informed consent Not applicable.

References

Aksoy S, Avunduk S, Gavcar EG, Büber A (2025) Evaluation of social media privacy, internet addiction in adolescents who are victims of sexual abuse and digital awareness in their parents. J Psychiatr Res. https://doi.org/10.1016/j.jpsychires.2025.01.030

Balakrishnan V, Khan S, Arabnia HR (2020) Improving cyberbullying detection using Twitter users' psychological features and machine learning. Comput Secur 90:101710. https://doi.org/10.1016/j.cose.2019.101710

Bermeo A, Acosta-Urigüen MI, Orellana M, Valdivieso Albán SI (2022) Cyberbullying Through the Lens of Data Science. In: Herrera-Tapia, J., Rodriguez-Morales, G., Fonseca C., E.R., Berrezueta-Guzman, S.

(eds) Information and Communication Technologies. TICEC 2022. Communications in Computer and Information Science, vol 1648. Springer, Cham. https://doi.org/10.1007/978-3-031-18272-3_16

Corizzo R, Zdravevski E, Russell M, Vagliano A, Japkowicz N (2020) Feature extraction based on word embedding models for intrusion detection in network traffic. J Surveill Secur Saf. https://doi.org/10.20517/jsss.2020.15

Dadvar M, Eckert K (2018) Cyberbullying detection in social networks using deep learning based models; a reproducibility study. arXiv preprint arXiv:1812.08046. https://doi.org/10.48550/arXiv.1812.08046

Dadvar M, Eckert K (2020) Cyberbullying Detection in Social Networks Using Deep Learning Based Models. In: Song, M., Song, IY., Kotsis, G., Tjoa, A.M., Khalil, I. (eds) Big Data Analytics and Knowledge Discovery. DaWaK 2020. Lecture Notes in Computer Science(), vol 12393. Springer, Cham. https://doi.org/10.1007/978-3-030-59065-9_20

I Deliu, C Leichter and K Franke, (2018) "Collecting Cyber Threat Intelligence from Hacker Forums via a Two-Stage, Hybrid Process using Support Vector Machines and Latent Dirichlet Allocation," 2018 IEEE International Conference on Big Data (Big Data), Seattle, WA, USA, 2018, pp. 5008–5013, https://doi.org/10.1109/BigData.2018.8622469.

Dewani A, Memon MA, Bhatti S (2021) Cyberbullying detection: advanced preprocessing techniques & deep learning architecture for Roman Urdu data. J Big Data 8:160. https://doi.org/10.1186/s40537-021-00550-7

Graney-Ward C, Issac B, Ketsbaia L et al (2022) Detection of cyberbullying through BERT and weighted ensemble of classifiers. TechRxiv. https://doi.org/10.3627/techrxiv.17705009.v1

Haydar N, Dhannoon BN (2025) GloVe vs. Fasttext embedding in detecting arabic cyberbullying using sequence learning classifiers. In AIP Conference Proceedings (Vol. 3169, No. 1). AIP Publishing. https://doi.org/10.1063/5.0255958

Islam MF, Hasan J, Islam MA, Dewan P, Rahman MS (2025) BanglaLem: a transformer-based bangla lemmatizer with an enhanced dataset. Syst Soft Comput. https://doi.org/10.1016/j.sasc.2025.200244

Iwendi C, Srivastava G, Khan S et al (2023) Cyberbullying detection solutions based on deep learning architectures. Multimedia Syst 29:1839–1852. https://doi.org/10.1007/s00530-020-00701-5

Kalpani N, Rodrigo N, Seneviratne D et al (2025) Cutting-edge approaches in intrusion detection systems: a systematic review of deep learning, reinforcement learning, and ensemble techniques. Iran J Comput Sci. https://doi.org/10.1007/s42044-025-00246-8

S. Khalid, T. Khalil, and S. Nasreen, "A survey of feature selection and feature extraction techniques in machine learning," Proc 2014 Sci Inf Conf SAI 2014, no. August 2014, pp. 372–378, 2014, https://doi.org/10.1109/SAI.2014.6918213.

Kim B, Yuvaraj N, Sri Preethaa KR et al (2021) Surface crack detection using deep learning with shallow CNN architecture for enhanced computation. Neural Comput Applic 33:9289–9305. https://doi.org/10.1007/s00521-021-05690-8

V. Lampos, B. Zou, and I. J. Cox, "Enhancing feature selection using word embeddings: The case of flu surveillance," 26th Int. World Wide Web Conf. WWW 2017, no. January 2018, pp. 695–704, 2017, https://doi.org/10.1145/3038912.3052622.

Logeswari G, Thangaramya K, Selvi M et al (2025) An improved synergistic dual-layer feature selection algorithm with two type classifier for efficient intrusion detection in IoT environment. Sci Rep 15:8050. https://doi.org/10.1038/s41598-025-91663-z

Murshed BAH, Abawajy J, Mallappa S, Saif MAN, Al-Ariki HDE (2022) DEA-RNN: a hybrid deep learning approach for cyberbullying detection in twitter social media platform. IEEE Access 10:25857–25871. https://doi.org/10.1109/ACCESS.2022.3153675

Nabeel F, Iqbal K (2025) Privacy, data protection and cyber crimes: mapping perceptions of pakistani users. J Appl Sec Res 20(2):293–319. https://doi.org/10.1080/19361610.2024.2372989

Nahar V, Al-Maskari S, Li X, Pang C (2014) Semi-supervised Learning for Cyberbullying Detection in Social Networks. In: Wang, H., Sharaf, M.A. (eds) Databases Theory and Applications. ADC 2014. Lecture Notes in Computer Science, vol 8506. Springer, Cham. https://doi.org/10.1007/978-3-319-08608-8_14

Payette M, Abdul-Nour G, Meango TJM, Diago M, Côté A (2025) Leveraging failure modes and effect analysis for technical language processing. Mach Learn Knowl Extr 7(2):42. https://doi.org/10.3390/make7020042

Prabowo WA, Azizah F (2020) Sentiment analysis for detecting cyberbullying using TF-IDF and SVM. Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi) 4(6):1142–1148. https://doi.org/10.29207/resti.v4i6.2753

Rai A, Borah S (2025) Tokenization and stemming of limbu language. ACM Trans Asian Low-Resour Lang Inform Proc. https://doi.org/10.1145/3712018

Rios SA, Aguilera F, Nuñez-Gonzalez JD, Graña M (2019) Semantically enhanced network analysis for influencer identification in online social networks. Neurocomputing 326:71–81. https://doi.org/10.1016/j.neucom.2017.01.123

Simran K, Balakrishna P, Vinayakumar R, Soman KP (2020) Deep learning approach for enhanced cyber threat indicators in twitter stream. In: Thampi, S., Martinez Perez, G., Ko, R., Rawat, D. (eds) Security in Computing and Communications. SSCC 2019. Communications in Computer and Information Science, vol 1208. Springer, Singapore. https://doi.org/10.1007/978-981-15-4825-3_11

Singh P, Sharma G (2025) Comparative analysis of FastText and Doc2Vec for semantic feature oriented software defect prediction. In Intelligent Computing and Communication Techniques (pp. 99–106). CRC Press.

Tarade R, Das S (2025) Cybersecurity in the Age of AI-Enhancing Defences for Today's Threats. Critical Phishing Defense Strategies and Digital Asset Protection, 309.

Teepapal T (2025) AI-driven personalization: unraveling consumer perceptions in social media engagement. Comput Hum Behav 165:108549. https://doi.org/10.1016/j.chb.2024.108549

Wu C, Gao Y, Guo J (2025) Semi-supervised decision-level fusion bag-of-words for human action recognition. Int J Mach Learn Cyber. https://doi.org/10.1007/s13042-025-02667-0

Yazdinejad A, Kazemi M, Parizi RM, Dehghantanha A, Karimipour H (2023) An ensemble deep learning model for cyber threat hunting in industrial internet of things. Digital Commun Netw 9(1):101–110. https://doi.org/10.1016/j.dcan.2022.09.008

Zhang Y (2025) Social Network User Profiling for Anomaly Detection Based on Graph Neural Networks. arXiv preprint arXiv:2503.19380. https://doi.org/10.48550/arXiv.2503.19380

Zhao R, Mao K (2017) Cyberbullying detection based on semantic-enhanced marginalized denoising auto-encoder. IEEE Trans Affect Comput 8(3):328–339. https://doi.org/10.1109/TAFFC.2016.2531682

Zhou H, Li Z, Pan L, Tian J, Zhu L (2025) Online chatter detection via lightweight deep learning framework with efficient signal pre-processing. Mech Syst Signal Process 223:111882. https://doi.org/10.1016/j.ymssp.2024.111882

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.